BUILD & OPERATE

DE10-Nano Robotic Arm Receiver

Complete step-by-step guide and engineering record



Quartus + Platform Designer • Six PWM channels
Firebase Realtime Database • Angstrom Linux
ESP32/W5500 gateway • One-command startup

Document date: 27 October 2026
Includes editable source package and archived RBF

Verified outcome: user-confirmed movement of all six joints and working startup after reboot. Consistent 50-150 ms latency is not established.

Contents

DE10-Nano robotic arm receiver 3

1. Final system and reproduction prerequisites 4

2. Wiring and power connections 6

3. PWM specification and register map 8

4. Create the Quartus project 9

5. Pin assignments, constraints and compilation 12

6. Create and configure Firebase 14

7. Laptop-only Firebase acceptance test 16

8. PuTTY and first board inspection 17

9. Load the new FPGA image for the first test 18

10. Verify registers before motor power 19

11. Internet, transfer and old Linux compatibility 20

12. Receiver operation and staged tests 22

13. Direct Firebase-console control of all six joints 23

14. Make startup persistent 24

15. Daily operation - final procedure 26

16. Change history and problems resolved 27

17. Troubleshooting by layer 28

18. Results, limits and acceptance record 29

19. Sources, archived files and reproduction notes 30

Appendix A. Final source-file inventory 31

Appendix B. All explicit physical pin assignments 32

Appendix C. Complete top-level and PWM HDL 35

Appendix D. Complete board receiver 39

Appendix E. Complete laptop test program 44

Appendix F. Final launcher 48

DE10-Nano robotic arm receiver

Complete build, wiring, Firebase and startup guide

Reconstruction of the RX work completed on 25-27 October 2026. Linux host, Quartus Prime 25.1 Standard/Lite, Cyclone V HPS, six FPGA PWM channels, Firebase Realtime Database and ESP32/W5500 internet gateway.

This guide records the final working configuration and the corrections that led to it. The user confirmed movement of all six servos through Firebase and operation of the one-command launcher after a normal reboot. It is not a guarantee for a different board, mechanical assembly, SD image or network. Follow the checkpoints rather than treating compilation as proof of hardware operation.

The primary daily command on the RX board is:

```
python /home/root/start_rx.py
```

This is manual Firebase-console control. The transmitter kit is not required and its implementation is outside this report.

How to use this report

Read chapters 1-3 before connecting power. Follow chapters 4-14 in order for a fresh build. Chapter 15 is the daily operating procedure. Chapters 16-18 explain the corrections, diagnosis and evidence. The appendices contain final source listings, all explicit pin locations and machine-readable parameter references.

A companion ZIP is attached inside this PDF and also supplied separately. It contains editable Quartus source files, the tested RBF, generated handoff files, receiver programs, configuration templates and SHA-256 checksums. In a PDF reader that supports attachments, open the paperclip/Attachments panel and save RX_Companion.zip. If your viewer hides attachments, use the separately supplied ZIP. Do not retype long HDL files from PDF when the source package is available.

Private account passwords, Wi-Fi credentials and authentication tokens are deliberately excluded. Use your own values in the clearly marked configuration templates.

1. Final system and reproduction prerequisites

1.1 Signal path



Firebase Console or laptop test publisher -> Firebase Realtime Database -> internet -> ESP32 Wi-Fi/NAT -> W5500 Ethernet -> DE10-Nano HPS Linux -> lightweight HPS-to-FPGA bridge -> six 32-bit PIO registers -> six 50 Hz PWM generators -> servos.

The ESP32/W5500 is the network gateway, not the PWM controller. The HPS authenticates to Firebase and processes the event stream. The FPGA generates the actual servo timing independently of Python scheduling. A USB-UART connection provides PuTTY access; Ethernet provides networking.

1.2 What is required

Item	Configuration used / requirement
RX board	Terasic DE10-Nano, Cyclone V 5CSEBA6U23I7; exact PCB revision was not recorded
Host computer	Linux laptop; recorded user home /home/ooh
FPGA tools	Quartus Prime 25.1std.0 Build 1129, Lite edition with Cyclone V support and Platform Designer
Boot media	Existing working DE10-Nano SD image: Angstrom v2014.12, Linux 4.5.0-00185-g3bb556b, U-Boot/SPL 2013.01.01 built October 2016
RX software	Python 2.7.3, BusyBox devmem, curl 7.37.1 with GnuTLS 3.3.5; /dev/mem available as root
Laptop software	Python 3.8+ and requests for optional test publisher; SSH/SCP; PuTTY
Servos	Three MG996R: base, shoulder, elbow; three MG90S: pitch, roll, claw
Servo supply	Separate regulated 5 V supply; current rating and distribution must support the actual motors and load
Network option A	ESP32/W5500 already configured as Wi-Fi-to-Ethernet NAT router
Network option B	Linux laptop Wi-Fi uplink with Ethernet connection sharing; fully usable alternative
Cables	USB-UART data cable, Ethernet cable, properly rated servo-power wiring, GPIO signal wiring

1.3 Important reproduction boundary

The original SD-card image and the ESP32 router firmware binaries are not part of the supplied project. You need a working compatible boot image, or a separately validated modern image with its own FPGA-loading procedure. This report does not create a new bootloader, device tree or Linux root filesystem from scratch.

The archived Quartus HPS configuration requests an 800 MHz MPU clock. The installed preloader reported 925 MHz MPU and 400 MHz DDR at boot. That is evidence that the legacy preloader controls the running HPS clocks; loading the RBF does not replace its initialization code. The existing system booted and moved all six servos, but this does not validate a newly generated preloader or every DE10-Nano DDR revision.

The observed board firmware performed DDR calibration successfully and reported 1 GiB. Verify the memory part, preset and handoff compatibility if reproducing on another hardware revision. Do not blindly copy DDR values to another board.

1.4 Where commands run

Label in this report	Run here
LAPTOP	Ordinary Linux terminal on the computer
RX / PuTTY	Linux shell on the DE10-Nano, normally root@socfpga
U-BOOT	SOCFPGA_CYCLONE5 prompt after interrupting autoboot
FIREBASE	Browser console for your Firebase project
QUARTUS	Quartus / Platform Designer GUI on the laptop

Commands in one environment are not interchangeable. For example, scp is run on the laptop; devmem is run on the board; fpga load is run in U-Boot.

2. Wiring and power connections

2.1 Complete six-servo signal mapping

The mapping below was checked against the final rx_arm.qsf and rx_arm.v. Header numbering agrees with Terasic's JP1/GPIO_0 diagram [R1].

Joint / software key	HDL output	JP1 physical pin	FPGA ball	Signal level
Base / base	SERVO_PWM[0]	1	V12	3.3 V LVTTL
Shoulder / shoulder	SERVO_PWM[1]	2	E8	3.3 V LVTTL
Elbow / elbow	SERVO_PWM[2]	3	W12	3.3 V LVTTL
Wrist pitch / pitch	SERVO_PWM[3]	4	D11	3.3 V LVTTL
Wrist roll / roll	SERVO_PWM[4]	5	D8	3.3 V LVTTL
Claw / claw	SERVO_PWM[5]	6	AH13	3.3 V LVTTL
Common ground	Board GND	12 (or 30)	Not an FPGA signal	Ground
FPGA clock	FPGA_CLK1_50	On-board oscillator	V11	50 MHz
External logic reset	RESET_N / KEY0	On-board key	AH17	Active low

Locate the JP1 marking and the pin-1 marker on the physical board. Do not choose a header by its position in a rotated photo. Number paired rows from pin 1:

```
JP1 / GPIO_0, numbered from the pin-1 end
 1 Base       2 Shoulder
 3 Elbow      4 Wrist pitch
 5 Wrist roll 6 Claw
 7            8
 9           10
11 +5 V      12 GND  <-- common ground only
```

The diagram is a numbering guide, not a guarantee of left/right orientation in your viewing direction. GPIO_0[5] means the sixth GPIO signal; its physical header location is pin 6. These are not the same as FPGA ball numbers.

2.2 Power distribution

For each servo, connect signal to the assigned JP1 pin, positive to the external regulated +5 V bus and ground to the external supply negative bus. Connect that negative bus to JP1 pin 12. Power the DE10-Nano through its normal board supply. The motor current should return to the servo supply, not through a thin jumper on the FPGA board.

Typical servo colours are red = positive, brown/black = ground and orange/yellow/white = signal. Confirm the actual connector rather than trusting colour alone. Do not join the external +5 V bus to JP1's power rail or any signal pin. Do not power six motors through the GPIO header. Avoid using a small solderless breadboard as the main high-current motor distribution bus.

The user used an external 5 V supply and common ground. Its current rating was not recorded; therefore no numerical six-motor supply rating has been validated here. Size it for simultaneous load/stall demand from the actual servos and use suitable distribution/fusing. A voltage-only label does not establish sufficient capacity.

Keep servo power off while connecting wires or loading the FPGA. Support the arm: disabling PWM can remove holding torque and let the shoulder/elbow fall. Have a physical power cutoff available. Start with one servo, use small changes and avoid sustained buzzing or mechanical stops.

2.3 Network and serial wiring

USB-UART: laptop USB to the board's UART-to-USB connector, not the USB-Blaster programming connector. Use PuTTY at 115200 baud, 8 data bits, no parity, 1 stop bit, no flow control.

Final network: RX RJ45 -> Ethernet cable -> W5500 RJ45; W5500 SPI -> ESP32; ESP32 Wi-Fi -> hotspot/router -> internet. No extra ESP32 GPIO connection to the RX board is required for the network path.

The supplied ESP32-W5500 reference document lists the following wiring. It was not rebuilt or re-flashed during this RX session; verify it against the firmware and exact breakout before constructing a new gateway.

W5500 signal	ESP32 pin named in supplied reference
CS / NSS	GPIO 5
SCK	GPIO 18
MISO	GPIO 19
MOSI	GPIO 23
GND	GND
VCC	3V3 for the documented 3.3 V breakout
RST, INT	Left unconnected in that reference; board-specific biasing/driver requirements apply

Some W5500 modules have regulators and different power labels. Use the module's specification. SPI logic is 3.3 V. The old router document names bootloader.bin, partitions.bin and esp32_ethernet_router.bin, but the actual binaries/partition table were not available for this report. Do not reuse its application flash offset without matching those files. A preconfigured router or laptop sharing avoids that missing dependency.

3. PWM specification and register map

3.1 User-supplied pulse calibration

These are the user's calibrated project ranges, not universal MG996R/MG90S manufacturer limits. The software angle is a linear command scale; it is not measured joint-position feedback. The reported claw opening was 55 mm, but this session did not measure it.

Joint	Servo	Minimum pulse	Maximum pulse	Minimum ticks	Maximum ticks
Base	MG996R	600 us	2400 us	30000	120000
Shoulder	MG996R	600 us	2400 us	30000	120000
Elbow	MG996R	600 us	2600 us	30000	130000
Wrist pitch	MG90S	420 us	2380 us	21000	119000
Wrist roll	MG90S	420 us	2380 us	21000	119000
Claw	MG90S	400 us	1200 us	20000	60000

At 50 MHz: one tick = 20 ns; pulse ticks = pulse microseconds x 50. A 20 ms frame contains 1,000,000 ticks. PWM frequency is 50 Hz.

```
ticks = round(min_ticks + (max_ticks - min_ticks) * angle / 180)
valid command angle: finite number, 0 through 180
channel disabled: write 0 ticks
```

A software angle of 0 is a valid minimum pulse when enable is true. It is not the same as disabling the channel. In the FPGA, zero or an out-of-range tick command disables the PWM output at the next frame boundary. Reset disables it directly. Do not shift counts right by four: the PIOs are 32 bits wide and store full tick counts.

3.2 Memory-mapped PIOs

Lightweight HPS-to-FPGA physical base = 0xFF200000. Each PIO's data register is its base offset, with a 16-byte address span.

PIO instance	Export	Offset	CPU physical address
pio_pwm_base	pwm_base	0x00	0xFF200000
pio_pwm_shoulder	pwm_shoulder	0x10	0xFF200010
pio_pwm_elbow	pwm_elbow	0x20	0xFF200020
pio_pwm_pitch	pwm_pitch	0x30	0xFF200030
pio_pwm_roll	pwm_roll	0x40	0xFF200040
pio_pwm_claw	pwm_claw	0x50	0xFF200050

The six outputs are pulse-width commands feeding HDL, not six PIO pins directly toggled by Linux. The HPS uses 32-bit little-endian register writes. The receiver reads back each set of writes. Readback proves register access, not correct physical PWM wiring.

4. Create the Quartus project

4.1 Recommended reproducible route

Extract the companion ZIP. Copy its Quartus folder to a working directory such as /home/ooh/Desktop/RX_Arm_Fresh. Open rx_arm.qpf in Quartus 25.1. The final rx_soc.qsys, top-level HDL, PWM module, QSF and SDC are included; generated IP must be regenerated before compiling. This preserves exact parameters instead of relying on screenshots or remembered defaults.

Do not copy old outputs/rx_arm.v from earlier development work; use the final file in the companion. It contains the corrected generated HPS port names.

4.2 Building the same project from a blank GUI

In File -> New Project Wizard, choose an empty project directory, project name rx_arm and top-level entity rx_arm. Select Cyclone V device 5CSEBA6U23I7. Add rx_arm.v and servo_pwm.v when available. Open Tools -> Platform Designer and save the system as rx_soc.qsys.

The actual working laptop installed Quartus under /home/ooh/altera_lite/25.1std/quartus/bin. Substitute your installation path when using commands below. The companion filenames are relative to the Quartus project directory; do not nest the entire project one directory too deep.

4.3 Clock and HPS IP

Add a Clock Source named clk_0. Set its input frequency to 50,000,000 Hz. Export clk_in as clk and clk_in_reset as reset. Add the Arria V/Cyclone V Hard Processor System IP as hps_0.

In FPGA Interfaces, enable only the lightweight HPS-to-FPGA bridge (32 bits). Set full HPS-to-FPGA and FPGA-to-HPS widths to unused/0; remove FPGA-to-HPS SDRAM ports. MPU events/general-purpose/debug/trace features are not needed in this design. Export h2f_reset as hps_fpga_reset. Leave the HPS memory and peripheral conduits exported as memory and hps_io.

4.4 HPS Peripheral Pins tab

Peripheral	Pin selection	Mode
EMAC0	Unused	N/A
EMAC1	HPS I/O Set 0	RGMII
NAND, QSPI	Unused	N/A
SDIO	HPS I/O Set 0	4-bit Data
USB0	Unused	N/A
USB1	HPS I/O Set 0	SDR, PHY clock output mode in GUI
UART0	HPS I/O Set 0	No Flow Control
I2C0	HPS I/O Set 0	I2C
Remaining UART, I2C, SPI, CAN, TRACE	Unused	N/A

USB1 Set 1 caused a conflict with EMAC1 Set 0 during the original build. USB1 Set 0 resolved it. Do not accept a red pin-mux conflict simply because HDL can be generated. No HPS GPIO/loan-I/O signals were enabled in the archived system.

4.5 HPS Clocks tab

Input EOSC1 = 25 MHz and EOSC2 = 25 MHz. FPGA-to-HPS SDRAM/peripheral PLL reference-clock enables are off. Use default MPU clock (desired 800 MHz). The saved system uses L4 MP/SP desired clocks 100 MHz, configuration clock 100 MHz, SDMMC 200 MHz, EMAC1 250 MHz, SPI master 200 MHz and USB MP 200 MHz. These are HPS tool settings, not the FPGA PWM input clock.

Peripheral PLL source is EOSC1. SDMMC/NAND source is the peripheral NAND/SDMMC clock; L4 MP/SP source is peripheral base clock. Keep the exact archived Qsys for all PLL dividers; the full exported parameter CSV is supplied. Do not force the FPGA clock source to 800 MHz or 100 MHz: the PWM counter and lightweight bridge are clocked from clk_0 at 50 MHz.

4.6 SDRAM tab - final saved values

The final Qsys is the source of truth for this build. The earlier generic defaults of 8-bit memory, 300 MHz memory clock and 125 MHz reference were replaced. The final saved tIS is 150 ps, not a remembered or proposed value from an earlier draft.

Setting	Final saved value
Protocol / vendor / format	DDR3 / Micron / discrete
Memory clock / reference	400 MHz / 25 MHz
Memory device speed grade	800 MHz
Supply / I/O	1.5 V DDR3 / SSTL-15
Data width / DQS groups	32 bits / 4 groups of 8
Chip selects / clocks / clock enables	1 / 1 / 1
Row / column / bank address widths	15 / 10 / 3
DM pins / differential DQS	Enabled / enabled
CAS latency / write CAS	7 / 7
Drive strength / nominal ODT	RZQ/6 / RZQ/6
Dynamic ODT	Off
Burst length / burst type	OTF (chop 4 or 8) / sequential
DLL / power-down DLL selection	Enabled / DLL off
Auto self-refresh / temperature	Manual / normal
Mirror addressing	0

The Qsys includes MEM_DEVICE=MISSING_MODEL and synthesis calibration-related parameters. Do not read these as proof of a validated physical memory preset. This design reused a working board preloader; another board/image requires its own DDR validation.

4.7 SDRAM timing and board settings

Timing parameter	Value	Timing parameter	Value
tIS	150 ps	tIH	140 ps
tDS	30 ps	tDH	65 ps
tDQSQ	125 ps	tQH	0.38 cycles
tDQSCK	255 ps	tDQSS	0.25 cycles
tQSH	0.4 cycles	tDSH / tDSS	0.2 / 0.2 cycles
tINIT	500 us	tMRD	4 cycles
tRAS	36 ns	tRCD / tRP	13.125 / 13.125 ns
tREFI	7.8 us	tRFC	300 ns
tWR	15 ns	tWTR	4 cycles
tFAW	45 ns	tRRD / tRTP	7.5 / 7.5 ns

Board setup/hold derating and channel-ISI methods are AUTO. Final board-skew parameters (ns): maximum CK delay 0.03; maximum DQS delay 0.02; CK-DQS minimum difference 0.06 and maximum 0.12; within-DQS skew 0.01; between-DQS skew 0.06; DQ-to-DQS difference 0.05; address/command skew 0.02; address/command-to-CK difference 0.01. Eye-reduction/delta-arrival values are zero in the saved system. These are recorded values, not newly measured PCB delays.

4.8 Add the six PIOs and connect the system

Add six PIO (Parallel I/O) components with names and exports from chapter 3. For each choose width 32, Output direction, reset value 0, no interrupts/edge capture and no separate bit-modifying output registers. Do not use a 16-bit PIO: the elbow maximum of 130000 needs more than 16 bits.

Connect clk_0.clk to hps_0.h2f_lw_axi_clock and all six PIO clk inputs. Connect clk_0.clk_reset to each PIO reset input. Connect hps_0.h2f_lw_axi_master to every PIO s1 port. Assign/lock the offsets 0x00, 0x10, 0x20, 0x30, 0x40 and 0x50 to the correct joints. Export the external_connection conduits as pwm_base, pwm_shoulder, pwm_elbow, pwm_pitch, pwm_roll and pwm_claw.

The top-level HDL combines RESET_N and exported hps_fpga_reset, synchronizes release with four FPGA clock edges, and feeds the resulting active-low reset into the Qsys reset input and all six PWM modules. Do not tie the HPS reset output to a constant or leave it disconnected. The final saved system connection/export tables are also included as CSV files in the companion.

Validate: no errors. The warning that the HPS model no longer supports FPGA-bridge simulation was observed. It is a simulation-model limitation, not a requirement to disable the real bridge. Save and Generate HDL for synthesis, Verilog. Regenerate after export or connection changes.

4.9 Add generated files and top-level logic

Back in Quartus, add rx_soc/synthesis/rx_soc.qip through Project -> Add/Remove Files. Add rx_arm.v and servo_pwm.v and set rx_arm as top level. Use the final complete source in the appendix/package; HPS and DDR ports must all be connected.

The PWM module latches a new count at the start of a frame to avoid truncating a live pulse mid-frame. Its counter resets to 999999 so the first released clock can begin a new frame. Six independent instances use the joint-specific min/max values. The register reset is zero, so booting the FPGA does not intentionally enable a servo.

5. Pin assignments, constraints and compilation

5.1 FPGA signal pins

Use the eight clock/reset/servo locations in chapter 2, with 3.3-V LVTTL I/O standard. In the QSF use the exact node spelling and square brackets:

```
set_location_assignment PIN_V11 -to FPGA_CLK1_50
set_location_assignment PIN_AH17 -to RESET_N
set_location_assignment PIN_V12 -to SERVO_PWM[0]
set_location_assignment PIN_E8 -to SERVO_PWM[1]
set_location_assignment PIN_W12 -to SERVO_PWM[2]
set_location_assignment PIN_D11 -to SERVO_PWM[3]
set_location_assignment PIN_D8 -to SERVO_PWM[4]
set_location_assignment PIN_AH13 -to SERVO_PWM[5]
```

These are QSF lines, not shell commands. A failed intermediate script wrote literal braces around target names. Quartus ignored those LOCATION assignments. The corrected QSF has bare node names as shown above. Full pin assignments, including the 72 explicit DDR locations, are in Appendix B and the supplied QSF.

5.2 HPS peripheral names and DDR constraints

The final top-level HPS peripheral ports use generated names such as hps_io_hps_io_emac1_inst_TX_CLK. Earlier conventional HPS_* peripheral aliases did not match generated SDC port queries. The top-level ports were renamed to the generated spelling and 3.3-V LVTTL assignments added for all 36 exported HPS peripheral ports.

The DDR ports retain HPS_DDR3_* names and have explicit locations. Generated DDR assignments set SSTL-15 CLASS I or differential equivalents, termination and package-skew settings. Do not change DDR ports to 3.3 V. Dedicated HPS peripheral routing is generated from the HPS pin mux, not assigned to spare GPIO balls.

5.3 Timing file

Save rx_arm.sdc and add it to the project:

```
create_clock -name FPGA_CLK1_50 -period 20.000 [get_ports {FPGA_CLK1_50}]
derive_clock_uncertainty
```

The braces here are valid SDC/Tcl syntax. That is a different parser context from the erroneous QSF literal-brace node names above. The generated QIP also brings in IP-specific timing constraints.

5.4 Run synthesis, apply DDR script, then full compile

Run Processing -> Start -> Start Analysis & Synthesis. After successful synthesis, run on the LAPTOP from the project directory:

```
cd /home/ooh/Desktop/RX_Arm_Fresh
/home/ooh/altera_lite/25.1std/quartus/bin/quartus_sta \
  -t rx_soc/synthesis/submodules/hps_sdram_p0_pin_assignments.tcl \
  rx_arm
```

Expected message: evaluation of the Tcl script was successful. This generates electrical/termination constraints but did not, in this build, provide all board-specific DDR locations. Keep the explicit location assignments in the supplied final QSF. Reopen/reload project settings if necessary before full compilation.

Run Processing -> Start Compilation. Check Fitter pin assignments and TimeQuest, not just Analysis & Synthesis. Reject unresolved critical LOCATION/RZQ errors, unassigned physical pins and unexpected port-name mismatches. The final archive used 116 fitted pins. Generated HPS HDL warnings about unused signals, dangling pins and truncations were reviewed in context; they are not a blanket permission to ignore warnings in user logic.

Warning 18236 (processor count unspecified) affects compilation resource scheduling. Optionally set NUM_PARALLEL_PROCESSORS to a sensible count for the host. It is not a servo-control fault. Do not edit generated HPS HDL just to suppress warnings; regeneration would discard the edits.

5.5 Actual timing status

The final saved report has positive slack for the reported constrained checks, with a smallest displayed worst-case slack of 0.076 ns. It also lists two unconstrained HPS clocks (USB CLK and I2C SCL), one unconstrained input (RESET_N) and six unconstrained servo outputs. This is not a claim of complete timing sign-off. External asynchronous PWM endpoints and HPS clock modeling need appropriate interpretation; do not add arbitrary false paths simply to make the report green.

5.6 Create the RBF

On the LAPTOP:

```
cd /home/ooh/Desktop/RX_Arm_Fresh
/home/ooh/altera_lite/25.1std/quartus/bin/quartus_cpf \
  -c output_files/rx_arm.sof output_files/rx_arm.rbf
ls -l output_files/rx_arm.rbf
sha256sum output_files/rx_arm.rbf
```

The recorded RBF is 7,007,204 bytes. A rebuilt file need not have an identical size/hash; compare the actual file transferred to the board with the build you intend to deploy. Do not assume any nonempty RBF contains this design. The companion manifest identifies the archived binary. The .sof is for JTAG workflows; the .rbf is used by the verified U-Boot loader here.

6. Create and configure Firebase

6.1 Project, database and accounts

In Firebase Console create a project. The session project was robotic-arm-rx. Create Realtime Database (not Firestore); the project used asia-southeast1, Singapore. Copy the exact database URL displayed by the console. The recorded URL was:

```
https://robotic-arm-rx-default-rtdb.asia-southeast1.firebasedatabase.app
```

For a new project use its own URL. Under Authentication -> Sign-in method, enable Email/Password. Create a dedicated RX reader and a separate test-writer account. These are Firebase email/password accounts; an email ending in gmail.com does not mean Google OAuth sign-in. The password is the Firebase account password, not necessarily the Gmail account password.

Record both UIDs. Deleting and recreating an account changes its UID, even if the email is reused. In this session the final reader UID was Mfsh7prA78ZqW2oAAuVklFtZOkp1 and writer UID SnGoXE5FygZc9vGac5DgFWt3VNf1. These are historical identifiers, not values to copy into a new project.

Under Project settings -> General -> Your apps, add a Web app such as rx-test. Hosting is not required. Select SDK setup/configuration -> Config and copy apiKey from firebaseConfig. You do not need to install the JavaScript SDK for the Python REST client. The UID and API key are different values. REST email/password authentication is documented in [R2].

6.2 Publish the database rules

Replace RX_UID and WRITER_UID with your actual current account UIDs, then publish:

```
{
  "rules": {
    ".read": false,
    ".write": false,
    "rx_arm": {
      "command": {
        ".read": "auth != null && (auth.uid === 'RX_UID' || auth.uid === 'WRITER_UID')",
        ".write": "auth != null && auth.uid === 'WRITER_UID'",
        ".validate": "newData.hasChildren(['seq', 'sent_ms', 'angles', 'enable'])"
      }
    }
  }
}
```

These are the bring-up rules used in the project, with UID placeholders. They require the main fields but do not enforce all numeric limits server-side; the receiver validates them. Do not change .read/.write globally to true to fix a credential mismatch. Firebase Console edits made by a project administrator are distinct from the reader account's application permissions.

6.3 Create the initial database object

Import the companion firebase_initial_data.json at the database root, or construct the equivalent object below. Importing at the wrong level creates a doubled rx_arm path. All enable values start false; 90 is only an illustrative stored value, not a universal mechanically safe position.

```
{
  "rx_arm": {
    "command": {
      "seq": 1,
      "sent_ms": 0,
      "angles": {
        "base": 90, "shoulder": 90, "elbow": 90,
        "pitch": 90, "roll": 90, "claw": 90
      },
      "enable": {
        "base": false, "shoulder": false, "elbow": false,
        "pitch": false, "roll": false, "claw": false
      }
    }
  }
}
```

The initial sent_ms=0 is intentionally stale in normal mode. It is accepted only by manual-console mode; the test publisher creates fresh timestamps. Use JSON numbers and booleans, not strings such as "90" or "true". Keep all six entries even when only one servo is enabled.

6.4 Private client configuration

Create rx_config.json from the template, filling your own values:

```
{
  "database_url": "https://YOUR_DATABASE.firebasedatabase.app",
  "api_key": "YOUR_WEB_API_KEY",
  "email": "YOUR_RX_EMAIL",
  "password": "YOUR_RX_FIREBASE_PASSWORD"
}
```

Create writer_config.json with the same URL/key but the writer account's email/password. On both host and board protect these files:

```
chmod 600 rx_config.json writer_config.json
```

On the RX board only rx_config.json is needed; do not copy writer credentials there. Neither credentials nor tokens are included in the companion archive.

7. Laptop-only Firebase acceptance test

This stage was completed before the physical board was available. It isolates authentication/schema issues from FPGA or servo faults. Copy software/rx_receiver.py from the companion into a host folder and run on the LAPTOP:

```
mkdir -p /home/ooh/Desktop/RX_Cloud_Test
cd /home/ooh/Desktop/RX_Cloud_Test
python3 -m venv .venv
.venv/bin/python -m pip install requests
# Place rx_receiver.py and the two private configs in this folder.
.venv/bin/python rx_receiver.py listen --config rx_config.json
```

Leave that dry-run listener running. In a second LAPTOP terminal:

```
cd /home/ooh/Desktop/RX_Cloud_Test
.venv/bin/python rx_receiver.py send --config writer_config.json --seconds 3
```

Expected accepted ticks: [0, 0, 0, 0, 0, 0]. Then test conversion only:

```
.venv/bin/python rx_receiver.py send --config writer_config.json \
  --joint roll --angle 90 --seconds 3
.venv/bin/python rx_receiver.py send --config writer_config.json --seconds 3
```

Expected roll ticks 70000, followed by all zeros. The helper enables only the selected joint and disables the others. Do not use it on an unsupported physical arm that needs other joints energized for support.

A first "command is not an object" means the path is empty or has an unexpected type. A first stale rejection usually means the stored snapshot is old. Fresh publisher commands should then pass. Do not remove the freshness check merely to conceal clock or network problems; manual-console mode is a separate deliberate operating mode.

8. PuTTY and first board inspection

8.1 Serial setup and logging

Use PuTTY serial at 115200, 8-N-1, flow control None. Identify the laptop UART device with ls -l /dev/serial/by-id/ or inspect /dev/ttyUSB* before and after connecting. If necessary on Ubuntu add the host user to dialout and log out/in. Log in with credentials for your SD image; this board used root.

On Linux PuTTY, Shift+Insert worked for pasting. Selection copying/right-click did not work reliably for this user. Session -> Logging -> All session output, saved to /home/ooh/Desktop/rx_putty.log, allowed review without clipboard copying. Avoid logging commands that print passwords/tokens. A log is a record, not a source of shell instructions; type only the command blocks, not displayed output such as 0x00000000 or the chat word "done".

8.2 Identify the board environment

Run in RX/PuTTY:

```
uname -a
cat /etc/os-release
cat /proc/device-tree/model
printf '\n'
cat /proc/cmdline
python3 --version
python --version
ip addr
ip route
date -u
ls -l /dev/fpga* /sys/class/fpga_manager/ /sys/class/fpga-bridge/
mount
lsblk
```

The actual system had no python3, no /dev/fpga*, an empty FPGA-manager class, no FPGA-bridge class and BusyBox ip without the -br option. It had Python 2.7.3, gcc, /sbin/devmem and root access to /dev/mem. These findings led to the legacy curl-based receiver and U-Boot programming route. Do not use a generic /dev/fpga0 command or an assumed sysfs firmware attribute on this image.

SD layout observed: mmcblk0p1 approximately 819.2 MiB (FAT boot files), mmcblk0p2 approximately 1 GiB (root filesystem), mmcblk0p3 approximately 1 MiB (boot-related raw partition). Do not format or repurpose partition 3. The root filesystem had about 310 MiB free at inspection.

8.3 Inspect the actual boot partition

```
mkdir -p /mnt/sd_boot
mount -o ro /dev/mmcblk0p1 /mnt/sd_boot
ls -lah /mnt/sd_boot
strings /mnt/sd_boot/u-boot.scr
```

If already mounted, inspect its mount options instead of mounting it twice. The files observed were rx_arm.rbf, soc_system.rbf, socfpga.dtb, u-boot.scr and zImage. /boot in the Linux root filesystem contained older kernel files and was not this mounted FAT partition.

The existing script loaded soc_system.rbf, ran bridge_enable_handoff, mmcload and mmcboot. Copying a second RBF onto the card did not activate it. Several RBFs can be stored, but this full-configuration workflow has one active FPGA image at a time.

9. Load the new FPGA image for the first test

First copy output_files/rx_arm.rbf to the FAT boot partition using a correctly identified SD-card reader/mount, or SCP plus a controlled copy on the board. Preserve the old image. Keep the reader program stopped and motor power off while changing FPGA configuration. Verify transferred file size/hash. Never write the RBF to the raw SD disk.

For the recorded legacy U-Boot, enter reboot in Linux, then interrupt "Hit any key to stop autoboot". At SOCFPGA_CYCLONE5, inspect before loading:

```
printenv bootcmd
printenv fpgadata
printenv bridge_enable_handoff
printenv mmcload
printenv mmcboot
help fpga
```

Recorded fpgadata = 0x2000000. bootcmd = run callscript; run mmcload; run mmcboot. The board's existing bridge macro performed its handoff writes and helper call. The recorded handoff values were fpgaintf=0, fpga2sdram=0, axibridge=0 and l3remap=0x19. These observations are specific to this bootloader, not generic addresses to overwrite on any system.

Load the file at the U-BOOT prompt:

```
if fatload mmc 0:1 ${fpgadata} rx_arm.rbf; then fpga load 0 ${fpgadata} ${filesize}; fi
```

The recorded file read was 7,007,204 bytes and fpga load returned without error. fpga info 0 describes the device, not proof of servo function. Stop if reading/programming fails; do not continue to register accesses.

After successful loading run, one at a time:

```
run bridge_enable_handoff
run mmcload
run mmcboot
```

The bridge helper returned rc=0x0; mmcload read zImage and socfpga.dtb; Linux reached root login. Do not use boot here because its callscript path would load the older soc_system.rbf until persistence is installed. Do not use saveenv for this initial test.

10. Verify registers before motor power

The following addresses apply only to this Cyclone V design and the matching loaded image. A disabled/misconfigured FPGA bridge can hang the CPU; a timeout cannot reliably undo a stalled bus transaction.

In RX/PuTTY, with servo power off:

```
devmem 0xFF706000 32
devmem 0xFFD0501C 32
```

Observed values: 0x00000004 (FPGA manager user mode) and 0x00000000 (bridge resets released). Register definitions are in the HPS reference [R4]. Then read the six data registers:

```
devmem 0xFF200000 32
devmem 0xFF200010 32
devmem 0xFF200020 32
devmem 0xFF200030 32
devmem 0xFF200040 32
devmem 0xFF200050 32
```

All six initially read zero. Check write/readback on roll, still with motor power off:

```
devmem 0xFF200040 32 70000
devmem 0xFF200040 32
devmem 0xFF200040 32 0
devmem 0xFF200040 32
```

Expected 0x00011170 then 0x00000000. This passed. Do not run devmem writes concurrently with the receiver; the receiver's software lock cannot prevent an unrelated devmem process from writing.

11. Internet, transfer and old Linux compatibility

11.1 ESP32/W5500 route used in final operation

The recorded RX address was 192.168.4.2/24, gateway 192.168.4.1, with DNS resolving through configured servers. This address was obtained by the existing networking setup; it is not a universal static address. Do not copy the older router document's 192.168.4.50 static configuration over a working DHCP setup.

In RX/PuTTY:

```
cat /sys/class/net/eth0/carrier
ip addr show eth0
ip route
cat /etc/resolv.conf
ping -c 3 google.com
date -u
```

carrier=1 means link detected. NO-CARRIER/0 initially occurred simply because the cable was not connected. A successful ping proves network reachability, not Firebase authentication or servo latency. The log showed 100 Mbps link through W5500 and successful DNS/ping.

11.2 Laptop sharing for transfers or as the main gateway

Connect the RX Ethernet cable directly to the laptop. Keep laptop Wi-Fi internet active. In wired connection settings set IPv4 to Shared to other computers, save and reconnect. The board automatically obtained 10.42.0.209 with gateway 10.42.0.1 during this session. Use the actual address from ip addr, not that example blindly.

SCP to 192.168.4.2 stalled when the board was behind the ESP32 NAT and the laptop had no direct route. Outbound internet access does not imply inbound SSH access. Switching temporarily to laptop Ethernet solved the transfer without changing Firebase. Return to W5500 afterward; keep USB-UART connected and avoid rebooting before boot persistence is installed.

11.3 HTTPS probe and time

```
curl --connect-timeout 10 --max-time 20 \
  -o /tmp/firebase_probe.json \
  -w '\nHTTP status: %{http_code}\n' \
  'https://YOUR_DATABASE.firebasedatabase.app/rx_arm/command.json'
cat /tmp/firebase_probe.json
```

Use your real URL. HTTP 401/Permission denied is expected without a token and confirmed successful HTTPS contact in this project. Do not use -k to bypass certificate verification. Correct the board's UTC clock and CA trust if HTTPS fails. Existing time services brought the board date forward; a fixed historical date is not a permanent clock-synchronization solution.

11.4 Why the board receiver differs from the laptop program

No Python 3 was installed and the local opkg catalog returned no python3 packages. This does not prove every online feed is unavailable; it means a suitable interpreter was not present/listed. Rather than installing packages from obsolete feeds, the board program was adapted to Python 2.7 using curl subprocesses for HTTPS/SSE. The existing curl 7.37.1/GnuTLS stack passed the Firebase HTTPS test. This is a legacy compatibility solution, not a claim that this old OS is a current secure deployment platform.

The board script uses only standard Python modules. Verify them:

```
python - <<'PY'
import json, mmap, struct, subprocess, threading
import Queue, fcntl, argparse, select
print("RX compatibility check: PASS")
PY
```

The actual board passed. Do not run the Python 3 requests-based laptop script on Python 2.7.

11.5 Install the final consolidated program

From the companion, transfer software/rx_receiver_legacy.py and your private reader rx_config.json. Example LAPTOP commands while directly connected (substitute your IP/source directory):

```
scp -O -o ConnectTimeout=10 rx_receiver_legacy.py \
  root@10.42.0.209:/home/root/
scp -O -o ConnectTimeout=10 rx_config.json \
  root@10.42.0.209:/home/root/
```

The -O option selects legacy SCP protocol for this older server. The host-key prompt is normal on first contact; verify the board endpoint before accepting. Modern SSH warned about lack of post-quantum key exchange, but both file transfers completed. It was not a transfer failure.

If keeping W5500 connected, use PuTTY to create the file with cat > /home/root/rx_receiver_legacy.py <<'PY', paste the complete final source in Appendix D, then put PY alone on a new line. Prefer the supplied plain-text file as the copy source. Compile and check the help before using hardware mode.

```
cd /home/root
chmod 600 rx_config.json
python -m py_compile rx_receiver_legacy.py
python rx_receiver_legacy.py listen --help
```

The final help must list --manual-console. The older transferred version lacked that option and produced "unrecognized arguments"; it was patched through PuTTY. A fresh installation from this companion already includes the change and needs no patch.

12. Receiver operation and staged tests

12.1 Board dry run

Start on RX/PuTTY:

```
python -u /home/root/rx_receiver_legacy.py listen \
  --config /home/root/rx_config.json
```

No --hardware means no /dev/mem access. Send fresh disabled commands from the laptop test helper. The actual board logged zero ticks and command ages of 322.8, 103.2, 109.6, 132.9 and 134.5 ms. After verifying, press Ctrl+C before any local register test or another receiver process.

12.2 Local claw test used in this project

Only the claw was initially powered. With motor power off, write:

```
devmem 0xFF200050 32 40000
devmem 0xFF200050 32
```

Readback 0x00009C40 means 40000 ticks, an 800 us pulse and 90 on the user's claw command scale. With the gripper clear, briefly power the servo and observe. The user reported a response. Switch motor power off and clear:

```
devmem 0xFF200050 32 0
```

For a different arm, choose a known-safe command based on its calibration and mounting. Do not assume this project's unusually short claw pulse range suits every MG90S. A scope/logic-analyzer check of a 20 ms frame and the intended high time is recommended before powering a new assembly; waveform capture was not recorded in this session.

12.3 Timestamped Firebase hardware test

Start RX hardware mode, with motor power initially off:

```
python -u /home/root/rx_receiver_legacy.py listen \
  --config /home/root/rx_config.json --hardware
```

Send from LAPTOP:

```
.venv/bin/python rx_receiver.py send --config writer_config.json \
  --joint claw --angle 90 --seconds 5
```

Expect [0, 0, 0, 0, 0, 40000]. Power the already checked claw and send a small change:

```
.venv/bin/python rx_receiver.py send --config writer_config.json \
  --joint claw --angle 100 --seconds 3
```

Expect claw 42222 ticks; the user reported slight movement. Finish by turning motor power off and sending an all-disabled command:

```
.venv/bin/python rx_receiver.py send --config writer_config.json --seconds 3
```

All-zero tick output was recorded. Stopping the publisher alone does not disable the receiver's held target.

13. Direct Firebase-console control of all six joints

13.1 Why a separate mode was added

Normal mode expects an increasing seq and a sent_ms timestamp no older than 2000 ms and no more than 250 ms in the future. Editing only angles in the browser leaves both fields unchanged, so a normal receiver rejects the command or ignores its sequence.

Manual-console mode explicitly bypasses timestamp/sequence freshness by validating a local copy with a synthetic current timestamp and sequence zero. It still validates all angles and boolean enable fields, converts to joint-specific pulse counts, rejects invalid input and reads back hardware writes. It applies a changed tick vector and suppresses duplicate vectors. The stored Firebase seq/sent_ms fields are not rewritten by the reader.

In this mode the startup snapshot can be old and still be applied. Network interruption holds the last targets; reconnect can accept a changed stored target. There is no independent FPGA communication watchdog. Normal Ctrl+C/SIGTERM cleanup attempts to zero all registers; SIGKILL, power failure or a hung bus cannot guarantee cleanup. This policy was chosen for bring-up, not certified fail-safe robotics.

13.2 Start manual mode

With the arm supported and motor power off, stop the old receiver with Ctrl+C. In Firebase set all six enable values to false. Then start:

```
python -u /home/root/rx_receiver_legacy.py listen \
  --config /home/root/rx_config.json --hardware --manual-console
```

The startup text "disconnect policy=hold last target" describes behavior; it does not report that the internet is disconnected. Likewise "Listening: HARDWARE" only proves startup progressed to hardware access, not that Firebase login/stream is complete. A manual ticks line confirms acceptance of a command. The consolidated source may additionally print a manual-mode notice.

13.3 Edit the database

In Firebase Console -> Realtime Database -> Data -> rx_arm -> command:

Joint	Target field	Enable field
Base	angles/base	enable/base
Shoulder	angles/shoulder	enable/shoulder
Elbow	angles/elbow	enable/elbow
Wrist pitch	angles/pitch	enable/pitch
Wrist roll	angles/roll	enable/roll
Claw	angles/claw	enable/claw

Set each target to a mechanically appropriate number first, then set the corresponding enable to true. Move a few degrees at a time during initial verification. Multiple joints may remain enabled simultaneously. Disabling any joint removes PWM/holding torque; support the mechanism. All six angle/enable entries must remain present, including disabled joints. Do not run the laptop sender at the same time as console edits because it overwrites the full command object.

The final tests showed nonzero commands and changes for all six positions in the tick vector. The user confirmed every servo moved correctly. No transmitter kit was connected. Console-edit responsiveness was subjectively reported as low delay; manual mode has no meaningful source timestamp and therefore does not measure latency.

14. Make startup persistent

14.1 Two separate things must persist

The receiver file on Linux and the FPGA image selected by U-Boot are independent. A Python launcher cannot correct a wrong RBF loaded at boot. Initially rx_arm.rbf existed next to soc_system.rbf, but the boot script still selected the latter. Rebooting therefore undid the manual image selection.

The final solution kept the known boot script, preserved its old image, and copied the tested RX image under the name soc_system.rbf. This was done only after proving the exact existing loader/bridge path and testing the image on hardware. It is not a universal procedure for other SD images.

14.2 Install the working RBF under the boot filename

First support the arm, turn motor power off and stop the receiver. Prefer a full backup of the SD card before boot changes. Ensure rx_arm.rbf on its FAT partition is the tested build. In RX/PuTTY paste:

```
(
set -e
mkdir -p /mnt/sd_boot
if ! grep -q ' /mnt/sd_boot ' /proc/mounts; then
    mount /dev/mmcblk0p1 /mnt/sd_boot
fi
mount -o remount,rw /mnt/sd_boot
cd /mnt/sd_boot
test -s rx_arm.rbf
test -s soc_system.rbf
if [ ! -e soc_system.before_rx.rbf ]; then
    cp soc_system.rbf soc_system.before_rx.rbf
    cmp soc_system.rbf soc_system.before_rx.rbf
fi
cp rx_arm.rbf soc_system.new.rbf
cmp rx_arm.rbf soc_system.new.rbf
mv soc_system.new.rbf soc_system.rbf
sync
cmp rx_arm.rbf soc_system.rbf
mount -o remount,ro /mnt/sd_boot
echo "SUCCESS: RX image installed for automatic boot."
)
```

The block stops on failure, does not replace an existing backup, and compares the copied file. It is not power-loss-proof on FAT; keep power stable. The log showed an unclean-unmount warning on the FAT volume, followed by successful comparisons. A filesystem check while the partition is unmounted remains appropriate maintenance. Do not run a repair against a mounted boot filesystem.

14.3 Create the one-command Python launcher

Paste in RX/PuTTY:

```
cat > /home/root/start_rx.py <<'PY'
#!/usr/bin/env python
import os
import sys
os.chdir("/home/root")
os.execv(sys.executable, [
    sys.executable,
    "-u",
    "/home/root/rx_receiver_legacy.py",
    "listen",
    "--config", "/home/root/rx_config.json",
    "--hardware",
    "--manual-console"
])
PY
python -m py_compile /home/root/start_rx.py
ls -l /home/root/start_rx.py
sync
```

One attempt failed with "No such file or directory" because the launcher had not yet been created. Creating it and compiling it resolved that issue. This launcher does not install a boot service: the FPGA image loads automatically, while the user starts Python manually.

14.4 Verify normal reboot

Set all Firebase enable values false and leave motor power off. In RX/PuTTY:

```
sync
reboot
```

Do not interrupt autoboot. Verify the serial log reads soc_system.rbf and 7,007,204 bytes for this archived build, runs the bridge helper without error and reaches Linux login. Then run:

```
python /home/root/start_rx.py
```

The recorded normal reboot loaded the replaced soc_system.rbf, then Linux and the launcher ran. An Ethernet link interruption and short attempts stopped with Ctrl+C initially caused confusion. The user subsequently confirmed the one-command startup worked. Leave time for DHCP, Wi-Fi uplink and authentication; the login timeout is up to 30 seconds before a retry is reported.

14.5 Recovery if the boot image causes a problem

Keep servo power off. Interrupt U-Boot and load the preserved backup using the same verified loading sequence, substituting soc_system.before_rx.rbf for rx_arm.rbf. Boot the known Linux image. Do not start this RX receiver against an old unrelated FPGA design. To permanently restore, after verifying the backup exists, mount the FAT partition writable, copy the backup to a temporary file, compare it, rename it to soc_system.rbf, sync and remount read-only. Preserve rx_arm.rbf for diagnosis.

15. Daily operation - final procedure

1. Power the RX board and ESP32/W5500 gateway. Keep servo power off initially. Connect the gateway to its configured Wi-Fi internet source.

2. Open PuTTY on the USB-UART port, 115200/8-N-1/no flow control. Log in as root. No U-Boot intervention is normally needed.

3. Check the stored Firebase angle and enable values. Manual mode can apply these immediately; all-disabled values are appropriate when preparing an unsupported arm.

4. Run python /home/root/start_rx.py. Wait for a manual ticks line or diagnose an explicit connection error. Leave this process running.

5. Power the servos when the mechanism and stored targets are ready. Edit angles and enable values through Firebase.

6. When finished, support the arm, switch servo power off, then press Ctrl+C. For a board power-down, use poweroff and wait for shutdown before removing its supply. Avoid removing the SD card while mounted.

The FPGA image now persists across reboot; no repeated pin setup, compilation, RBF load or bridge command is required. ESP32 internet availability and the Linux network setup still have to recover normally. The receiver is not a background daemon; closing/killing its session may stop it. The startup line is a policy description, not a connection-status indicator.

16. Change history and problems resolved

Stage / symptom	Cause or evidence	Final correction
Earlier demo worked; retest no joints moved	Timeouts reported; CONF_DONE lit alone did not identify image or validate PWM	Rebuilt and verified each layer separately; no single historical root cause was proven
EMAC1 and USB1 conflict	USB1 selected Set 1	USB1 moved to HPS I/O Set 0
Generic SDRAM defaults	Initial 8-bit width, wrong clock/reference for intended board	Final Qsys recorded 32-bit DDR3, 400 MHz / 25 MHz plus explicit timings
HPS reset not exported initially	Top-level reset integration incomplete	Export hps_fpga_reset, regenerate and synchronize reset release in HDL
Generated model warnings	Unused/dangling HPS signals, simulator limitations	Reviewed context; used hardware tests; did not edit generated HDL blindly
DDR/RZQ location issues	Generated pin script insufficient for all physical locations	Added 72 explicit DDR locations from board mapping
Ignored LOCATION assignments	Literal braces in QSF target names	Wrote correct bare node names and recompiled
HPS SDC port mismatch	Conventional peripheral top-level aliases	Renamed 36 HPS ports to generated hps_io_hps_io_* names
Firebase sign-in HTTP 400	Invalid API key, later wrong account password	Copied Web app apiKey and corrected Firebase credentials
Database write HTTP 401	Writer config used reader account; recreated accounts changed UIDs	Correct writer email/password and both rule UID references
Missing / stale initial command	Empty database or old timestamp	Seed full schema and send fresh commands for normal mode
Board lacked Python 3	Angstrom Python 2.7; no Python 3 in local package catalog	Added curl-backed Python 2.7 board client
Ethernet NO-CARRIER	Cable initially absent	Connected cable; verified link, DHCP address and route
SCP stalled to 192.168.4.2	Laptop outside ESP32 NAT subnet	Temporary direct laptop Ethernet sharing; later PuTTY source patch
Reboot brought old behavior	Boot script named soc_system.rbf	Backed up old binary and installed tested image under that name
--manual-console unrecognized	Old receiver file remained on board	Installed equivalent manual-mode patch; consolidated final source included
Editing angles had stale/sequence issues	Timestamped command protocol rejects static console edits	Added explicit manual mode with changed-target detection
start_rx.py missing	Launcher creation block not yet executed	Created launcher, compiled it and tested startup
"HARDWARE; disconnect policy" mistaken for disconnection	Policy text interpreted as a fault	Explained status; checked actual commands and retry messages
PuTTY copy/paste unreliable	Linux selection/clipboard behavior	Shift+Insert for paste; file logging for inspection

17. Troubleshooting by layer

17.1 Authentication and database

HTTP 400 at sign-in is before the database rules. An invalid API key is not fixed by changing rules. Wrong password or disabled Email/Password provider must be corrected in Authentication. HTTP 401 from the unauthenticated curl probe is expected; HTTP 401 for a writer after successful login points to authorization/token/rules and needs its returned UID checked. Use current account UIDs after recreating users.

If needed run this on the LAPTOP to print only status/error/UID, not the token:

```
import json
import requests
with open("writer_config.json") as f:
    c = json.load(f)
r = requests.post(
    "https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword",
    params={"key": c["api_key"]},
    json={"email": c["email"], "password": c["password"],
          "returnSecureToken": True}, timeout=20)
d = r.json()
print("HTTP status:", r.status_code)
if r.status_code == 200:
    print("UID:", d["localId"])
else:
    print("Error:", d.get("error", {}).get("message"))
```

Do not paste the private config into shared logs. The legacy board client suppresses curl stderr to avoid leaking credentials and reports curl exit codes: 6 commonly indicates DNS resolution failure, 7 connection failure, 22 HTTP failure, 28 timeout and 60 certificate trust failure. Inspect network/date and the appropriate authenticated/unauthenticated probe rather than assuming every code is a password error.

17.2 Receiver starts but no target lines

Allow up to the configured login timeout. Check ip addr, ip route, DNS and date. Verify config URL/path and account permissions. Confirm manual mode is supported by listen --help and that no second hardware instance holds /tmp/rx_arm_mmio.lock. In manual mode unchanged tick vectors are suppressed; a lack of repeated lines is not automatically a failure. Logging is also rate-limited to roughly one line per second, while commands can be processed more frequently.

17.3 Firebase updates appear but servo does not move

Check enable is boolean true, angles are numeric, joint names match, and the right header is wired. Confirm physical servo power/common ground and supply voltage under load. Stop the receiver before devmem diagnostics. Verify the correct RBF, manager status and bridge release. Readback is not an oscilloscope: a wrong top-level pin can pass all memory tests. Observe PWM with a scope if available. A servo already at the commanded position may show little movement.

17.4 Network loss or reconnect

The receiver holds its last target and retries. Its SSE connection uses curl no-buffer; an idle-data timeout of 70 seconds triggers a reconnect, and curl has a 3000-second stream lifetime for periodic reauthentication. This is not a motion watchdog. Do not interpret continued holding torque as proof of live internet. The old log showed one idle timeout even while no new commands were being sent.

17.5 Boot failure / storage warning

A FAT unclean-unmount warning was observed. Back up the card and check the FAT partition offline using appropriate filesystem tools on a correctly identified device. Do not run blind fsck commands against the mounted root disk or assume the laptop SD reader is /dev/sdb. Keep stable power during replacement of the boot binary. The saved backup supports rollback; it does not replace a full SD-card backup.

18. Results, limits and acceptance record

Check	Evidence / outcome
Quartus build	User completed synthesis/full build; final files and reports retained; critical pin errors corrected
RBF transfer/loading	7,007,204 bytes read in U-Boot; no reported programming error
FPGA/bridge state	Manager status 0x4; bridge reset 0x0
PIO reset	Six zero readbacks
PIO write/read	Roll 70000 -> 0x11170, then zero
Local claw	40000 -> 0x9C40; user observed response
Board cloud dry run	Fresh disabled commands accepted through Firebase
Cloud claw motion	40000 then 42222 ticks; user observed slight movement
All six manual control	Log vectors changed for all channels; user reported all servos moving correctly
Normal boot	Replaced soc_system.rbf loaded by existing script; launcher started; user confirmed working
Latency target	Not established consistently; no physical end-to-end timing measurement

Normal-mode samples included 91.9-134.5 ms command ages, startup/other samples around 259-505 ms, and a spike of 998.9 ms. These are sparse log samples, not a distribution or percentile benchmark. They depend on writer/RX clock alignment, exclude physical servo travel, and are not hard real-time bounds. Console mode deliberately bypasses timestamps, so its printed tick updates cannot be used to calculate command age.

FPGA frame-boundary latching adds up to about 20 ms after a register change. Six software register writes are sequential, not a single atomic six-axis commit; writes crossing a frame boundary can update channels in different frames. There is no encoder feedback, trajectory interpolation, acceleration limit, independent watchdog, certified emergency-stop function or full long-duration/repeated-power-cycle qualification in this implementation.

For a new reproduction, complete: build checks; verified image and bridge; all-zero/reset and readback tests; one-servo local tests; authenticated dry run; small-angle cloud tests for each joint; explicit disable; supported-arm disconnect/reconnect test; and a normal cold boot with the intended gateway. Record the actual supply, SD-image identity and board revision. Never substitute a promise of "no problems" for those acceptance checks.

19. Sources, archived files and reproduction notes

[R1] Terasic DE10-Nano User Manual, mirrored by FPGA Academy: https://fpgacademy.org/Downloads/DE10_Nano_User_Manual.pdf . Used to cross-check GPIO/DDR board mapping; final values also read from the actual QSF.

[R2] Firebase Auth REST API: https://firebase.google.com/docs/reference/rest/auth/ . Email/password sign-in and ID-token authentication reference.

[R3] Firebase Realtime Database REST streaming: https://firebase.google.com/docs/database/rest/retrieve-data . SSE put/patch event protocol used by the supplied implementations.

[R4] Cyclone V HPS Technical Reference Manual: https://www.intel.com/programmable/technical-pdfs/683126.pdf . FPGA-manager and bridge-reset register reference.

[R5] Quartus scripting reference: https://cdrdv2-public.intel.com/654662/tclscriptrefmnl.pdf . Programming-file conversion reference.

[R6] Terasic getting-started guide: https://www.terasic.com.tw/attachment/archive/1046/Getting_Started_Guide.pdf . Board setup/serial reference.

Primary project evidence: /home/ooh/Desktop/RX_Arm_Fresh (final Qsys, QSF, HDL, SDC, timing report, handoff and RBF); /home/ooh/Desktop/rx_putty.log (boot commands, OS, networking, register reads and manual target logs); receiver source files created during this session; user confirmations of motion and startup.

The supplied ESP32-W5500 ROUTER.pdf was used only for its documented gateway wiring and to identify missing firmware dependencies. Its older static-address/kernel details and fixed-date suggestions were not substituted for the observed RX system. Earlier thesis/report drafts were background, not the authority for the final compiled pinout or runtime configuration.

The final consolidated legacy source contains the same manual-mode control behavior installed by the PuTTY patch, plus a diagnostic manual-mode notice. It is a maintained source copy, not a fresh download of the board filesystem. The launcher is reconstructed exactly in function from the recorded creation command. The archived Qsys/QSF/HDL and RBF are direct copies of the final laptop project. No firmware changes were made to the user's board while authoring this report.

The archive includes the complete raw Qsys, QSF, all component parameter and connection CSVs, the existing hps_isw_handoff folder and SHA256SUMS.txt. Generated rx_soc synthesis IP, the full SD image and ESP32 firmware are not included. Regenerate the Qsys IP with the matching tool release. Appendices print the user HDL and both clients so the PDF remains useful without external links; prefer original plain-text sources for actual builds.

Appendix A. Final source-file inventory

The companion ZIP is also embedded as a PDF attachment. Use its text files for compilation and deployment. Long printed lines may wrap visually; that is not a change to the source.

File	Role
Quartus/rx_arm.qpf	Project identity
Quartus/rx_arm.qsf	All location, electrical, IP and project assignments
Quartus/rx_soc.qsys	Complete Platform Designer system
Quartus/rx_arm.v	HPS/DDR integration, reset synchronizer and six PWM instances
Quartus/servo_pwm.v	Frame-latched PWM generator
Quartus/rx_arm.sdc	50 MHz primary clock constraint
Quartus/output_files/rx_arm.rbf	Archived programmed FPGA image
Quartus/hps_isw_handoff	Generated handoff reference; not the installed bootloader
software/rx_receiver_legacy.py	Python 2.7/curl board receiver with manual mode
software/rx_receiver.py	Python 3/requests laptop test source and normal receiver
software/start_rx.py	Single-command manual-console launcher
software/*.example.json	Credential-free configuration/rules templates
software/firebase_initial_data.json	All-disabled initial database object
reference/*.csv	Exact component parameters, connections and exports
SHA256SUMS.txt	Integrity checks for package files

Archived RBF SHA-256:

```
7f46051a06929d0c0edd75bd5d7cfecb3ec8c6f3d2fe8a58a57433b89b5f4278
```

Appendix B. All explicit physical pin assignments

These are extracted directly from the final QSF. DDR connections are on-board traces, not external jumper-wire instructions. All DDR signals require the generated electrical constraints retained in that QSF. The 36 dedicated HPS peripheral signals are generated by the HPS pin mux and have 3.3-V LVTTL assignments; do not relocate them to GPIO.

Top-level signal	FPGA ball
FPGA_CLK1_50	V11
RESET_N	AH17
SERVO_PWM[5]	AH13
SERVO_PWM[4]	D8
SERVO_PWM[3]	D11
SERVO_PWM[2]	W12
SERVO_PWM[1]	E8
SERVO_PWM[0]	V12
HPS_DDR3_ADDR[0]	C28
HPS_DDR3_ADDR[1]	B28
HPS_DDR3_ADDR[2]	E26
HPS_DDR3_ADDR[3]	D26
HPS_DDR3_ADDR[4]	J21
HPS_DDR3_ADDR[5]	J20
HPS_DDR3_ADDR[6]	C26
HPS_DDR3_ADDR[7]	B26
HPS_DDR3_ADDR[8]	F26
HPS_DDR3_ADDR[9]	F25
HPS_DDR3_ADDR[10]	A24
HPS_DDR3_ADDR[11]	B24
HPS_DDR3_ADDR[12]	D24
HPS_DDR3_ADDR[13]	C24
HPS_DDR3_ADDR[14]	G23
HPS_DDR3_BA[0]	A27
HPS_DDR3_BA[1]	H25
HPS_DDR3_BA[2]	G25
HPS_DDR3_DM[0]	G28

Top-level signal	FPGA ball
HPS_DDR3_DM[1]	P28
HPS_DDR3_DM[2]	W28
HPS_DDR3_DM[3]	AB28
HPS_DDR3_DQ[0]	J25
HPS_DDR3_DQ[1]	J24
HPS_DDR3_DQ[2]	E28
HPS_DDR3_DQ[3]	D27
HPS_DDR3_DQ[4]	J26
HPS_DDR3_DQ[5]	K26
HPS_DDR3_DQ[6]	G27
HPS_DDR3_DQ[7]	F28
HPS_DDR3_DQ[8]	K25
HPS_DDR3_DQ[9]	L25
HPS_DDR3_DQ[10]	J27
HPS_DDR3_DQ[11]	J28
HPS_DDR3_DQ[12]	M27
HPS_DDR3_DQ[13]	M26
HPS_DDR3_DQ[14]	M28
HPS_DDR3_DQ[15]	N28
HPS_DDR3_DQ[16]	N24
HPS_DDR3_DQ[17]	N25
HPS_DDR3_DQ[18]	T28
HPS_DDR3_DQ[19]	U28
HPS_DDR3_DQ[20]	N26
HPS_DDR3_DQ[21]	N27
HPS_DDR3_DQ[22]	R27
HPS_DDR3_DQ[23]	V27
HPS_DDR3_DQ[24]	R26
HPS_DDR3_DQ[25]	R25
HPS_DDR3_DQ[26]	AA28
HPS_DDR3_DQ[27]	W26
HPS_DDR3_DQ[28]	R24

Top-level signal	FPGA ball
HPS_DDR3_DQ[29]	T24
HPS_DDR3_DQ[30]	Y27
HPS_DDR3_DQ[31]	AA27
HPS_DDR3_DQS_N[0]	R16
HPS_DDR3_DQS_N[1]	R18
HPS_DDR3_DQS_N[2]	T18
HPS_DDR3_DQS_N[3]	T20
HPS_DDR3_DQS_P[0]	R17
HPS_DDR3_DQS_P[1]	R19
HPS_DDR3_DQS_P[2]	T19
HPS_DDR3_DQS_P[3]	U19
HPS_DDR3_CAS_N	A26
HPS_DDR3_CKE	L28
HPS_DDR3_CK_N	N20
HPS_DDR3_CK_P	N21
HPS_DDR3_CS_N	L21
HPS_DDR3_ODT	D28
HPS_DDR3_RAS_N	A25
HPS_DDR3_RESET_N	V28
HPS_DDR3_WE_N	E25
HPS_DDR3_RZQ	D25

Appendix C. Complete top-level and PWM HDL

Source: Quartus/rx_arm.v.

```
// Receiver top level. Clock input must be 50 MHz.
// RESET_N is an active-low external reset, assigned to a board key later.
// SERVO_PWM[0..5]: base, shoulder, elbow, pitch, roll, claw.
// Board pin assignments and timing constraints must be added before fitting.
`default_nettype none
module rx_arm (
    input wire FPGA_CLK1_50,
    input wire RESET_N,
    output wire [5:0] SERVO_PWM,
    output wire  hps_io_hps_io_emac1_inst_TX_CLK,
    output wire  hps_io_hps_io_emac1_inst_TXD0,
    output wire  hps_io_hps_io_emac1_inst_TXD1,
    output wire  hps_io_hps_io_emac1_inst_TXD2,
    output wire  hps_io_hps_io_emac1_inst_TXD3,
    input wire  hps_io_hps_io_emac1_inst_RXD0,
    inout wire  hps_io_hps_io_emac1_inst_MDIO,
    output wire  hps_io_hps_io_emac1_inst_MDC,
    input wire  hps_io_hps_io_emac1_inst_RX_CTL,
    output wire  hps_io_hps_io_emac1_inst_TX_CTL,
    input wire  hps_io_hps_io_emac1_inst_RX_CLK,
    input wire  hps_io_hps_io_emac1_inst_RXD1,
    input wire  hps_io_hps_io_emac1_inst_RXD2,
    input wire  hps_io_hps_io_emac1_inst_RXD3,
    inout wire  hps_io_hps_io_sdio_inst_CMD,
    inout wire  hps_io_hps_io_sdio_inst_D0,
    inout wire  hps_io_hps_io_sdio_inst_D1,
    output wire  hps_io_hps_io_sdio_inst_CLK,
    inout wire  hps_io_hps_io_sdio_inst_D2,
    inout wire  hps_io_hps_io_sdio_inst_D3,
    inout wire  hps_io_hps_io_usb1_inst_D0,
    inout wire  hps_io_hps_io_usb1_inst_D1,
    inout wire  hps_io_hps_io_usb1_inst_D2,
    inout wire  hps_io_hps_io_usb1_inst_D3,
    inout wire  hps_io_hps_io_usb1_inst_D4,
    inout wire  hps_io_hps_io_usb1_inst_D5,
    inout wire  hps_io_hps_io_usb1_inst_D6,
    inout wire  hps_io_hps_io_usb1_inst_D7,
    input wire  hps_io_hps_io_usb1_inst_CLK,
    output wire  hps_io_hps_io_usb1_inst_STP,
    input wire  hps_io_hps_io_usb1_inst_DIR,
    input wire  hps_io_hps_io_usb1_inst_NXT,
    input wire  hps_io_hps_io_uart0_inst_RX,
    output wire  hps_io_hps_io_uart0_inst_TX,
    inout wire  hps_io_hps_io_i2c0_inst_SDA,
    inout wire  hps_io_hps_io_i2c0_inst_SCL,
    output wire [14:0] HPS_DDR3_ADDR,
    output wire [2:0] HPS_DDR3_BA,
    output wire  HPS_DDR3_CK_P,
    output wire  HPS_DDR3_CK_N,
    output wire  HPS_DDR3_CKE,
    output wire  HPS_DDR3_CS_N,
    output wire  HPS_DDR3_RAS_N,
    output wire  HPS_DDR3_CAS_N,
    output wire  HPS_DDR3_WE_N,
    output wire  HPS_DDR3_RESET_N,
    inout wire [31:0] HPS_DDR3_DQ,
    inout wire [3:0] HPS_DDR3_DQS_P,
    inout wire [3:0] HPS_DDR3_DQS_N,
    output wire  HPS_DDR3_ODT,
    output wire [3:0] HPS_DDR3_DM,
    input wire  HPS_DDR3_RZQ
);

    wire hps_reset_n;
    wire raw_reset_n = RESET_N & hps_reset_n;
    // Asynchronous assertion; release after four rising clock edges.
    (* altera_attribute = "-name SYNCHRONIZER_IDENTIFICATION FORCED" *)
    reg [3:0] reset_release = 4'b0000;
    always @(posedge FPGA_CLK1_50 or negedge raw_reset_n) begin
        if (!raw_reset_n)
            reset_release <= 4'b0000;
        else
            reset_release <= {reset_release[2:0], 1'b1};
    end
```

```
    wire system_reset_n = reset_release[3];

    wire [31:0] pwm_base_ticks;
    wire [31:0] pwm_shoulder_ticks;
    wire [31:0] pwm_elbow_ticks;
    wire [31:0] pwm_pitch_ticks;
    wire [31:0] pwm_roll_ticks;
    wire [31:0] pwm_claw_ticks;

    rx_soc u_soc (
        .clk_clk(FPGA_CLK1_50),
        .hps_fpga_reset_reset_n(hps_reset_n),
        .hps_io_hps_io_emac1_inst_TX_CLK(hps_io_hps_io_emac1_inst_TX_CLK),
        .hps_io_hps_io_emac1_inst_TXD0(hps_io_hps_io_emac1_inst_TXD0),
        .hps_io_hps_io_emac1_inst_TXD1(hps_io_hps_io_emac1_inst_TXD1),
        .hps_io_hps_io_emac1_inst_TXD2(hps_io_hps_io_emac1_inst_TXD2),
        .hps_io_hps_io_emac1_inst_TXD3(hps_io_hps_io_emac1_inst_TXD3),
        .hps_io_hps_io_emac1_inst_RXD0(hps_io_hps_io_emac1_inst_RXD0),
        .hps_io_hps_io_emac1_inst_MDIO(hps_io_hps_io_emac1_inst_MDIO),
        .hps_io_hps_io_emac1_inst_MDC(hps_io_hps_io_emac1_inst_MDC),
        .hps_io_hps_io_emac1_inst_RX_CTL(hps_io_hps_io_emac1_inst_RX_CTL),
        .hps_io_hps_io_emac1_inst_TX_CTL(hps_io_hps_io_emac1_inst_TX_CTL),
        .hps_io_hps_io_emac1_inst_RX_CLK(hps_io_hps_io_emac1_inst_RX_CLK),
        .hps_io_hps_io_emac1_inst_RXD1(hps_io_hps_io_emac1_inst_RXD1),
        .hps_io_hps_io_emac1_inst_RXD2(hps_io_hps_io_emac1_inst_RXD2),
        .hps_io_hps_io_emac1_inst_RXD3(hps_io_hps_io_emac1_inst_RXD3),
        .hps_io_hps_io_sdio_inst_CMD(hps_io_hps_io_sdio_inst_CMD),
        .hps_io_hps_io_sdio_inst_D0(hps_io_hps_io_sdio_inst_D0),
        .hps_io_hps_io_sdio_inst_D1(hps_io_hps_io_sdio_inst_D1),
        .hps_io_hps_io_sdio_inst_CLK(hps_io_hps_io_sdio_inst_CLK),
        .hps_io_hps_io_sdio_inst_D2(hps_io_hps_io_sdio_inst_D2),
        .hps_io_hps_io_sdio_inst_D3(hps_io_hps_io_sdio_inst_D3),
        .hps_io_hps_io_usb1_inst_D0(hps_io_hps_io_usb1_inst_D0),
        .hps_io_hps_io_usb1_inst_D1(hps_io_hps_io_usb1_inst_D1),
        .hps_io_hps_io_usb1_inst_D2(hps_io_hps_io_usb1_inst_D2),
        .hps_io_hps_io_usb1_inst_D3(hps_io_hps_io_usb1_inst_D3),
        .hps_io_hps_io_usb1_inst_D4(hps_io_hps_io_usb1_inst_D4),
        .hps_io_hps_io_usb1_inst_D5(hps_io_hps_io_usb1_inst_D5),
        .hps_io_hps_io_usb1_inst_D6(hps_io_hps_io_usb1_inst_D6),
        .hps_io_hps_io_usb1_inst_D7(hps_io_hps_io_usb1_inst_D7),
        .hps_io_hps_io_usb1_inst_CLK(hps_io_hps_io_usb1_inst_CLK),
        .hps_io_hps_io_usb1_inst_STP(hps_io_hps_io_usb1_inst_STP),
        .hps_io_hps_io_usb1_inst_DIR(hps_io_hps_io_usb1_inst_DIR),
        .hps_io_hps_io_usb1_inst_NXT(hps_io_hps_io_usb1_inst_NXT),
        .hps_io_hps_io_uart0_inst_RX(hps_io_hps_io_uart0_inst_RX),
        .hps_io_hps_io_uart0_inst_TX(hps_io_hps_io_uart0_inst_TX),
        .hps_io_hps_io_i2c0_inst_SDA(hps_io_hps_io_i2c0_inst_SDA),
        .hps_io_hps_io_i2c0_inst_SCL(hps_io_hps_io_i2c0_inst_SCL),
        .memory_mem_a(HPS_DDR3_ADDR),
        .memory_mem_ba(HPS_DDR3_BA),
        .memory_mem_ck(HPS_DDR3_CK_P),
        .memory_mem_ck_n(HPS_DDR3_CK_N),
        .memory_mem_cke(HPS_DDR3_CKE),
        .memory_mem_cs_n(HPS_DDR3_CS_N),
        .memory_mem_ras_n(HPS_DDR3_RAS_N),
        .memory_mem_cas_n(HPS_DDR3_CAS_N),
        .memory_mem_we_n(HPS_DDR3_WE_N),
        .memory_mem_reset_n(HPS_DDR3_RESET_N),
        .memory_mem_dq(HPS_DDR3_DQ),
        .memory_mem_dqs(HPS_DDR3_DQS_P),
        .memory_mem_dqs_n(HPS_DDR3_DQS_N),
        .memory_mem_odt(HPS_DDR3_ODT),
        .memory_mem_dm(HPS_DDR3_DM),
        .memory_oct_rzqin(HPS_DDR3_RZQ),
        .pwm_base_export(pwm_base_ticks),
        .pwm_claw_export(pwm_claw_ticks),
        .pwm_elbow_export(pwm_elbow_ticks),
        .pwm_pitch_export(pwm_pitch_ticks),
        .pwm_roll_export(pwm_roll_ticks),
        .pwm_shoulder_export(pwm_shoulder_ticks),
        .reset_reset_n(system_reset_n)
    );

    servo_pwm #(.MIN_TICKS(32'd30000), .MAX_TICKS(32'd120000)) u_pwm_base (
        .clk(FPGA_CLK1_50),
        .reset_n(system_reset_n),
        .pulse_ticks(pwm_base_ticks),
        .pwm_out(SERVO_PWM[0])
    );
```

```
    servo_pwm #(.MIN_TICKS(32'd30000), .MAX_TICKS(32'd120000)) u_pwm_shoulder (
        .clk(FPGA_CLK1_50),
        .reset_n(system_reset_n),
        .pulse_ticks(pwm_shoulder_ticks),
        .pwm_out(SERVO_PWM[1])
    );

    servo_pwm #(.MIN_TICKS(32'd30000), .MAX_TICKS(32'd130000)) u_pwm_elbow (
        .clk(FPGA_CLK1_50),
        .reset_n(system_reset_n),
        .pulse_ticks(pwm_elbow_ticks),
        .pwm_out(SERVO_PWM[2])
    );

    servo_pwm #(.MIN_TICKS(32'd21000), .MAX_TICKS(32'd119000)) u_pwm_pitch (
        .clk(FPGA_CLK1_50),
        .reset_n(system_reset_n),
        .pulse_ticks(pwm_pitch_ticks),
        .pwm_out(SERVO_PWM[3])
    );

    servo_pwm #(.MIN_TICKS(32'd21000), .MAX_TICKS(32'd119000)) u_pwm_roll (
        .clk(FPGA_CLK1_50),
        .reset_n(system_reset_n),
        .pulse_ticks(pwm_roll_ticks),
        .pwm_out(SERVO_PWM[4])
    );

    servo_pwm #(.MIN_TICKS(32'd20000), .MAX_TICKS(32'd60000)) u_pwm_claw (
        .clk(FPGA_CLK1_50),
        .reset_n(system_reset_n),
        .pulse_ticks(pwm_claw_ticks),
        .pwm_out(SERVO_PWM[5])
    );

endmodule
`default_nettype wire
```

PWM generator source

Source: Quartus/servo_pwm.v.

```
module servo_pwm #(
    parameter [31:0] MIN_TICKS = 32'd30000,
    parameter [31:0] MAX_TICKS = 32'd120000
)(
    input  wire        clk,
    input  wire        reset_n,
    input  wire [31:0] pulse_ticks,
    output wire        pwm_out
);

    // 50 MHz clock:
    // 1 tick = 20 ns
    // 1,000,000 ticks = 20 ms = 50 Hz servo frame
    localparam [19:0] FRAME_LAST = 20'd999999;

    reg [19:0] frame_count;
    reg [31:0] active_ticks;

    // Accept only pulse widths within this joint's limits.
    // Zero or an invalid value disables the channel.
    wire command_valid;

    assign command_valid =
        (pulse_ticks >= MIN_TICKS) &&
        (pulse_ticks <= MAX_TICKS);

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            frame_count <= FRAME_LAST;
            active_ticks <= 32'd0;
        end else begin
            if (frame_count == FRAME_LAST) begin
                frame_count <= 20'd0;

                // Apply changes only at the start of a frame.
                // This prevents changes from cutting a pulse short
                // or creating an extra pulse within the same frame.
                if (command_valid)
```

```
                    active_ticks <= pulse_ticks;
                else
                    active_ticks <= 32'd0;
            end else begin
                frame_count <= frame_count + 20'd1;
            end
        end
    end

    assign pwm_out =
        reset_n &&
        (active_ticks != 32'd0) &&
        ({12'd0, frame_count} < active_ticks);

endmodule
```

Appendix D. Complete board receiver

Source: software/rx_receiver_legacy.py.

```
#!/usr/bin/env python
"""Legacy RX receiver. Python 2.7/3 and curl; no third-party Python modules.
Never use hardware modes before the matching FPGA image and LW bridge are verified.
Network loss holds the last PWM targets. Normal process exit disables PWM.
"""
from __future__ import print_function
import argparse
import contextlib
import fcntl
import json
import math
import mmap
import os
import struct
import sys
import time
import subprocess
import select
import signal
try:
    from urllib.parse import urlparse, urlencode
except ImportError:
    from urlparse import urlparse
    from urllib import urlencode
try:
    INTEGER = (int, long)
    STRING = (basestring,)
except NameError:
    INTEGER = (int,)
    STRING = (str,)
NUMBER = INTEGER + (float,)
def finite(v):
    return not math.isnan(v) and not math.isinf(v)
def clock():
    return os.times()[4]
def log(message):
    print(message)
    sys.stdout.flush()


JOINTS = ('base', 'shoulder', 'elbow', 'pitch', 'roll', 'claw')
LIMITS = ((30000,120000), (30000,120000), (30000,130000),
          (21000,119000), (21000,119000), (20000,60000))
BASE = 0xFF200000
OFFSETS = tuple(i * 0x10 for i in range(6))


def ticks_for(index, angle):
    if isinstance(angle, bool) or not isinstance(angle, NUMBER):
        raise ValueError('angle must be numeric')
    if not finite(angle) or not 0 <= angle <= 180:
        raise ValueError('angle outside 0..180')
    lo, hi = LIMITS[index]
    return int(round(lo + (hi - lo) * angle / 180.0))


def decode_command(value, now_ms, max_age_ms=2000):
    if not isinstance(value, dict):
        raise ValueError('command is not an object')
    seq = value.get('seq')
    sent = value.get('sent_ms')
    if isinstance(seq, bool) or not isinstance(seq, INTEGER) or seq < 0:
        raise ValueError('invalid sequence')
    if isinstance(sent, bool) or not isinstance(sent, NUMBER) or not finite(sent):
        raise ValueError('invalid timestamp')
    age = now_ms - sent
    if age < -250 or age > max_age_ms:
        raise ValueError('stale command or clocks not synchronized')
    angles, enable = value.get('angles'), value.get('enable')
    if not isinstance(angles, dict) or not isinstance(enable, dict):
        raise ValueError('missing angles/enable objects')
    result = []
    for i, joint in enumerate(JOINTS):
        if type(enable.get(joint)) is not bool:
```

```
            raise ValueError('enable values must be booleans')
        tick = ticks_for(i, angles.get(joint))
        result.append(tick if enable[joint] else 0)
    return seq, age, result


def apply_event(state, kind, envelope):
    """Apply Firebase put/patch, including nested and multipath updates."""
    if not isinstance(envelope, dict) or not isinstance(envelope.get('path'), STRING):
        raise ValueError('malformed SSE event')
    def put(root, path, value):
        parts = [p for p in path.split('/') if p]
        if not parts:
            return value
        if not isinstance(root, dict):
            root = {}
        node = root
        for p in parts[:-1]:
            if not isinstance(node.get(p), dict):
                node[p] = {}
            node = node[p]
        if value is None:
            node.pop(parts[-1], None)
        else:
            node[parts[-1]] = value
        return root
    path, value = envelope['path'], envelope.get('data')
    if kind == 'put':
        return put(state, path, value)
    if kind == 'patch' and isinstance(value, dict):
        for key, v in value.items():
            state = put(state, path.rstrip('/') + '/' + key, v)
        return state
    raise ValueError('unsupported SSE update')


class Registers:
    def __init__(self):
        self.lock = open('/tmp/rx_arm_mmio.lock', 'a')
        fcntl.flock(self.lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
        fd = os.open('/dev/mem', os.O_RDWR | os.O_SYNC)
        try:
            self.mem = mmap.mmap(fd, 4096, flags=mmap.MAP_SHARED,
                                 prot=mmap.PROT_READ | mmap.PROT_WRITE, offset=BASE)
        finally:
            os.close(fd)

    def read(self):
        return [struct.unpack_from('<I', self.mem, o)[0] for o in OFFSETS]

    def write(self, values):
        if len(values) != 6:
            raise ValueError('six values required')
        for i, v in enumerate(values):
            if type(v) is not int or (v != 0 and not LIMITS[i][0] <= v <= LIMITS[i][1]):
                raise ValueError('invalid pulse count')
        for off, value in zip(OFFSETS, values):
            struct.pack_into('<I', self.mem, off, value)
        if self.read() != list(values):
            raise RuntimeError('PIO readback mismatch; stop and check FPGA/bridge')

    def close(self):
        self.mem.close()
        self.lock.close()


def load_config(path):
    with open(path) as f:
        c = json.load(f)
    u = urlparse(c['database_url'])
    if u.scheme != 'https' or not u.hostname or not u.hostname.endswith(('.firebasedatabase.app','.firebaseio.com')):
        raise ValueError('use the HTTPS Realtime Database URL')
    if u.query or u.fragment or u.username or u.path not in ('', '/'):
        raise ValueError('database_url must be the database root URL')
    for k in ('api_key', 'email', 'password'):
        if not c.get(k):
            raise ValueError('missing configuration: '+k)
    return c

def curl_quote(value):
```

```
    return '"' + value.replace('\\', '\\\\').replace('"', '\\"').replace('\n', '\\n').replace('\r', '\\r') + '"'


def start_curl(url, payload=None, stream=False):
    # Secrets travel over stdin, never command arguments or diagnostic output.
    config = ['url = ' + curl_quote(url), 'silent', 'fail',
              'connect-timeout = 10', 'max-time = ' + ('3000' if stream else '30')]
    if stream:
        config += ['no-buffer', 'header = "Accept: text/event-stream"']
    else:
        config += ['header = "Content-Type: application/json"',
                   'data = ' + curl_quote(json.dumps(payload))]
    null = open(os.devnull, 'wb')
    try:
        p = subprocess.Popen(['curl', '--config', '-'], stdin=subprocess.PIPE,
                             stdout=subprocess.PIPE, stderr=null, bufsize=0)
    finally:
        null.close()
    p.stdin.write(('\n'.join(config) + '\n').encode('utf-8'))
    p.stdin.close()
    p.stdin = None
    return p


def stop_curl(p):
    if p.poll() is None:
        p.terminate()
    p.wait()
    p.stdout.close()


def login(c):
    url = 'https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?' + urlencode({'key': c['api_key']})
    p = start_curl(url, {'email': c['email'], 'password': c['password'], 'returnSecureToken': True})
    try:
        data, _ = p.communicate()
        if p.returncode:
            raise RuntimeError('Firebase sign-in: curl exit %d' % p.returncode)
        result = json.loads(data.decode('utf-8'))
        return result['idToken']
    finally:
        stop_curl(p)


def events(p):
    pending = b''
    kind, data = None, []
    last_byte = clock()
    while True:
        ready, _, _ = select.select([p.stdout], [], [], 1)
        if not ready:
            if clock() - last_byte > 70:
                raise RuntimeError('Stream idle timeout')
            continue
        chunk = os.read(p.stdout.fileno(), 4096)
        if not chunk:
            code = p.wait()
            raise RuntimeError('Database stream ended: curl exit %d' % code)
        last_byte = clock()
        pending += chunk
        if len(pending) > 262144:
            raise RuntimeError('Stream line too large')
        while b'\n' in pending:
            line, pending = pending.split(b'\n', 1)
            line = line.rstrip(b'\r').decode('utf-8')
            if not line:
                if kind:
                    yield kind, '\n'.join(data)
                kind, data = None, []
            elif line.startswith('event:'):
                kind = line[6:].strip()
            elif line.startswith('data:'):
                data.append(line[5:].lstrip())
                if sum(len(x) for x in data) > 262144:
                    raise RuntimeError('Stream event too large')


def listen(args, c):
    regs = Registers() if args.hardware else None
    last_seq, last_log, delay = -1, 0, 1
```

```
    last_ticks = None
    try:
        if regs:
            regs.write([0]*6)
        log('Listening: %s; disconnect policy=hold last target; exit policy=disable' %
            ('HARDWARE' if regs else 'DRY RUN'))
        if args.manual_console:
            log('MANUAL CONSOLE: stored targets accepted, including after reconnect; timestamp/sequence checks
    bypassed')
        while True:
            p = None
            try:
                token = login(c)
                url = c['database_url'].rstrip('/') + '/rx_arm/command.json?' + urlencode({'auth':token})
                p = start_curl(url, stream=True)
                state = None
                for kind, payload in events(p):
                    if kind in ('cancel', 'auth_revoked'):
                        raise RuntimeError('Firebase stream ' + kind)
                    if kind not in ('put', 'patch'):
                        continue
                    delay = 1
                    try:
                        state = apply_event(state, kind, json.loads(payload))
                        candidate = state
                        if args.manual_console and isinstance(state, dict):
                            candidate = dict(state)
                            candidate['seq'] = 0
                            candidate['sent_ms'] = time.time()*1000
                        seq, age, ticks = decode_command(candidate, time.time()*1000)
                    except (ValueError, TypeError, KeyError, OverflowError):
                        if clock()-last_log >= 1:
                            log('Rejected: invalid/stale command; check timestamps and schema')
                            last_log = clock()
                        continue
                    if args.manual_console:
                        if ticks == last_ticks:
                            continue
                    elif seq <= last_seq:
                        continue
                    if regs:
                        regs.write(ticks)
                    last_seq = seq
                    last_ticks = ticks
                    if clock()-last_log >= 1:
                        log(('manual ticks=%s' % ticks) if args.manual_console else ('seq=%d age_ms=%.1f ticks=%s' %
    (seq, age, ticks)))
                        last_log = clock()
            except RuntimeError as e:
                if 'PIO readback' in str(e):
                    raise
                log('Connection issue: %s; holding targets; retry in %ds' % (e, delay))
            finally:
                if p is not None:
                    stop_curl(p)
            time.sleep(delay)
            delay = min(10, delay*2)
    finally:
        if regs:
            try:
                regs.write([0]*6)
            finally:
                regs.close()


def interrupted(signum, frame):
    raise KeyboardInterrupt()


def main():
    p = argparse.ArgumentParser(description=__doc__)
    sub = p.add_subparsers(dest='mode')
    l = sub.add_parser('listen')
    l.add_argument('--config', required=True)
    l.add_argument('--hardware', action='store_true')
    l.add_argument('--manual-console', action='store_true', help='Accept stored console targets without
    timestamp/sequence freshness checks')
    sub.add_parser('inspect')
    sub.add_parser('disable')
    args = p.parse_args()
```

```
    signal.signal(signal.SIGTERM, interrupted)
    if args.mode == 'listen':
        return listen(args, load_config(args.config))
    if args.mode not in ('inspect', 'disable'):
        p.error('select listen, inspect, or disable')
    with contextlib.closing(Registers()) as regs:
        if args.mode == 'inspect':
            log(str(dict(zip(JOINTS, regs.read()))))
        else:
            regs.write([0]*6)
            log('All PWM registers zero.')


if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        pass
    except Exception as e:
        # Never print arbitrary exception details that might include credentials.
        log('Stopped: ' + type(e).__name__)
        sys.exit(1)
```

Appendix E. Complete laptop test program

Source: software/rx_receiver.py.

```
#!/usr/bin/env python3
"""RX bring-up and Firebase SSE receiver. Python 3.8+, requests.
Never use hardware modes before the matching FPGA image and LW bridge are verified.
Network loss holds the last PWM targets. Normal process exit disables PWM.
"""
import argparse
import contextlib
import fcntl
import json
import math
import mmap
import os
import struct
import sys
import time
from urllib.parse import urlparse

JOINTS = ('base', 'shoulder', 'elbow', 'pitch', 'roll', 'claw')
LIMITS = ((30000,120000), (30000,120000), (30000,130000),
          (21000,119000), (21000,119000), (20000,60000))
BASE = 0xFF200000
OFFSETS = tuple(i * 0x10 for i in range(6))


def ticks_for(index, angle):
    if isinstance(angle, bool) or not isinstance(angle, (int, float)):
        raise ValueError('angle must be numeric')
    if not math.isfinite(angle) or not 0 <= angle <= 180:
        raise ValueError('angle outside 0..180')
    lo, hi = LIMITS[index]
    return int(round(lo + (hi - lo) * angle / 180.0))


def decode_command(value, now_ms, max_age_ms=2000):
    if not isinstance(value, dict):
        raise ValueError('command is not an object')
    seq = value.get('seq')
    sent = value.get('sent_ms')
    if isinstance(seq, bool) or not isinstance(seq, int) or seq < 0:
        raise ValueError('invalid sequence')
    if isinstance(sent, bool) or not isinstance(sent, (int,float)) or not math.isfinite(sent):
        raise ValueError('invalid timestamp')
    age = now_ms - sent
    if age < -250 or age > max_age_ms:
        raise ValueError('stale command or clocks not synchronized')
    angles, enable = value.get('angles'), value.get('enable')
    if not isinstance(angles, dict) or not isinstance(enable, dict):
        raise ValueError('missing angles/enable objects')
    result = []
    for i, joint in enumerate(JOINTS):
        if type(enable.get(joint)) is not bool:
            raise ValueError('enable values must be booleans')
        tick = ticks_for(i, angles.get(joint))
        result.append(tick if enable[joint] else 0)
    return seq, age, result


def apply_event(state, kind, envelope):
    """Apply Firebase put/patch, including nested and multipath updates."""
    if not isinstance(envelope, dict) or not isinstance(envelope.get('path'), str):
        raise ValueError('malformed SSE event')
    def put(root, path, value):
        parts = [p for p in path.split('/') if p]
        if not parts:
            return value
        if not isinstance(root, dict):
            root = {}
        node = root
        for p in parts[:-1]:
            if not isinstance(node.get(p), dict):
                node[p] = {}
            node = node[p]
        if value is None:
            node.pop(parts[-1], None)
```

```
        else:
            node[parts[-1]] = value
        return root
    path, value = envelope['path'], envelope.get('data')
    if kind == 'put':
        return put(state, path, value)
    if kind == 'patch' and isinstance(value, dict):
        for key, v in value.items():
            state = put(state, path.rstrip('/') + '/' + key, v)
        return state
    raise ValueError('unsupported SSE update')


class Registers:
    def __init__(self):
        self.lock = open('/tmp/rx_arm_mmio.lock', 'a')
        fcntl.flock(self.lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
        fd = os.open('/dev/mem', os.O_RDWR | os.O_SYNC)
        try:
            self.mem = mmap.mmap(fd, 4096, flags=mmap.MAP_SHARED,
                                 prot=mmap.PROT_READ | mmap.PROT_WRITE, offset=BASE)
        finally:
            os.close(fd)

    def read(self):
        return [struct.unpack_from('<I', self.mem, o)[0] for o in OFFSETS]

    def write(self, values):
        if len(values) != 6:
            raise ValueError('six values required')
        for i, v in enumerate(values):
            if type(v) is not int or (v != 0 and not LIMITS[i][0] <= v <= LIMITS[i][1]):
                raise ValueError('invalid pulse count')
        for off, value in zip(OFFSETS, values):
            struct.pack_into('<I', self.mem, off, value)
        if self.read() != list(values):
            raise RuntimeError('PIO readback mismatch; stop and check FPGA/bridge')

    def close(self):
        self.mem.close()
        self.lock.close()


def load_config(path):
    with open(path) as f:
        c = json.load(f)
    u = urlparse(c['database_url'])
    if u.scheme != 'https' or not u.hostname or not u.hostname.endswith(('.firebasedatabase.app','.firebaseio.com')):
        raise ValueError('use the HTTPS Realtime Database URL')
    if u.query or u.fragment or u.username or u.path not in ('', '/'):
        raise ValueError('database_url must be the database root URL')
    for k in ('api_key', 'email', 'password'):
        if not c.get(k):
            raise ValueError('missing configuration: '+k)
    return c


def login(session, c):
    r = session.post('https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword',
                     params={'key':c['api_key']}, json={'email':c['email'],
                     'password':c['password'],'returnSecureToken':True}, timeout=(10,20))
    if r.status_code != 200:
        raise RuntimeError('Firebase sign-in failed (HTTP %d)' % r.status_code)
    d = r.json()
    return d['idToken'], time.monotonic() + min(3000, int(d['expiresIn']) - 120)


def event_stream(response):
    kind, data = None, []
    # Small chunks avoid buffering small control messages waiting for 512 bytes.
    for line in response.iter_lines(chunk_size=1, decode_unicode=True):
        if not line:
            if kind:
                yield kind, '\n'.join(data)
            kind, data = None, []
        elif line.startswith('event:'):
            kind = line[6:].strip()
        elif line.startswith('data:'):
            data.append(line[5:].lstrip())
```

```
def listen(args, c):
    import requests
    regs = Registers() if args.hardware else None
    last_seq, last_log, delay = -1, 0.0, 1
    url = c['database_url'].rstrip('/') + '/rx_arm/command.json'
    try:
        if regs:
            regs.write([0]*6)
        print('Listening: '+('HARDWARE' if regs else 'DRY RUN')+
              '; disconnect policy=hold last target; exit policy=disable', flush=True)
        with requests.Session() as session:
            while True:
                try:
                    token, renew_at = login(session, c)
                    state = None
                    with session.get(url, params={'auth':token}, headers={'Accept':'text/event-stream'},
                                     stream=True, timeout=(10,65)) as response:
                        if response.status_code != 200:
                            raise RuntimeError('Database stream HTTP %d' % response.status_code)
                        response.encoding = 'utf-8'
                        delay = 1
                        for kind, payload in event_stream(response):
                            if time.monotonic() >= renew_at:
                                break
                            if kind in ('cancel','auth_revoked'):
                                raise RuntimeError('Firebase stream '+kind)
                            if kind not in ('put','patch'):
                                continue
                            try:
                                state = apply_event(state, kind, json.loads(payload))
                                seq, age, ticks = decode_command(state, time.time()*1000)
                            except (ValueError, TypeError, KeyError) as e:
                                if time.monotonic()-last_log >= 1:
                                    print('Rejected: '+str(e), flush=True); last_log=time.monotonic()
                                continue
                            if seq <= last_seq:
                                continue
                            if regs:
                                regs.write(ticks)
                            last_seq = seq
                            if time.monotonic()-last_log >= 1:
                                print('seq=%d age_ms=%.1f ticks=%s' % (seq, age, ticks), flush=True)
                                last_log = time.monotonic()
                except (requests.RequestException, RuntimeError) as e:
                    # Never print requests exception text: it can include the auth query token.
                    if isinstance(e, RuntimeError) and 'PIO readback' in str(e):
                        raise
                    label = str(e) if isinstance(e, RuntimeError) else type(e).__name__
                    print('Connection issue: %s; holding targets; retry in %ds' % (label,delay), flush=True)
                    time.sleep(delay); delay = min(10, delay*2)
    finally:
        if regs:
            try: regs.write([0]*6)
            finally: regs.close()


def send(args, c):
    """Laptop test source for RX acceptance; not the future TX-board program."""
    import requests
    if args.joint and args.angle is None:
        raise ValueError('--joint requires --angle')
    if args.joint:
        ticks_for(JOINTS.index(args.joint), args.angle)
    if args.seconds <= 0 or args.seconds > 60:
        raise ValueError('--seconds must be in (0,60]')
    with requests.Session() as s:
        token, _ = login(s, c)
        url = c['database_url'].rstrip('/') + '/rx_arm/command.json'
        end = time.monotonic()+args.seconds
        sequence = int(time.time()*1000000)
        while time.monotonic() < end:
            sequence += 1
            command = {'seq':sequence, 'sent_ms':int(time.time()*1000),
                       'angles':{j:args.angle if j==args.joint else 0 for j in JOINTS},
                       'enable':{j:j==args.joint for j in JOINTS}}
            r=s.put(url, params={'auth':token}, json=command, timeout=(10,20))
            if r.status_code != 200:
                raise RuntimeError('Test write failed (HTTP %d)' % r.status_code)
            time.sleep(0.1)
        print('Test source stopped; RX retains the last enabled target until disabled or stopped.')
```

```
def main():
    p=argparse.ArgumentParser(description=__doc__)
    sub=p.add_subparsers(dest='mode', required=True)
    l=sub.add_parser('listen'); l.add_argument('--config',required=True);
    l.add_argument('--hardware',action='store_true')
    s=sub.add_parser('send'); s.add_argument('--config',required=True); s.add_argument('--joint',choices=JOINTS)
    s.add_argument('--angle',type=float); s.add_argument('--seconds',type=float,default=3)
    sub.add_parser('inspect')
    sub.add_parser('disable')
    t=sub.add_parser('local-test');t.add_argument('--joint',choices=JOINTS,required=True)
    t.add_argument('--angle',type=float,required=True);t.add_argument('--seconds',type=float,default=3)
    args=p.parse_args()
    if args.mode in ('listen','send'):
        c=load_config(args.config)
        return listen(args,c) if args.mode=='listen' else send(args,c)
    if args.mode=='local-test' and not 0 < args.seconds <= 30:
        raise ValueError('test duration must be in (0,30]')
    with contextlib.closing(Registers()) as regs:
        if args.mode=='inspect':
            print(dict(zip(JOINTS,regs.read())))
        elif args.mode=='disable':
            regs.write([0]*6);print('All PWM registers zero.')
        else:
            values=[0]*6;values[JOINTS.index(args.joint)]=ticks_for(JOINTS.index(args.joint),args.angle)
            try:
                regs.write([0]*6);regs.write(values)
                print('PIO readback:',regs.read(),flush=True)
                time.sleep(args.seconds)
            finally:
                regs.write([0]*6)

if __name__=='__main__':
    try: main()
    except KeyboardInterrupt: pass
    except Exception as e:
        # Do not expose credentials through HTTP-library exceptions.
        print('Stopped: '+(str(e) if isinstance(e,(ValueError,RuntimeError,OSError)) else
    type(e).__name__),file=sys.stderr)
        sys.exit(1)
```

Appendix F. Final launcher

Source: software/start_rx.py.

```
#!/usr/bin/env python
import os
import sys
os.chdir("/home/root")
os.execv(sys.executable, [sys.executable, "-u",
    "/home/root/rx_receiver_legacy.py", "listen",
    "--config", "/home/root/rx_config.json",
    "--hardware", "--manual-console"])
```